

Introduction to Programming & Python

1. What is Programming?

Programming is the process of writing instructions that tell a computer what to do.

A computer cannot understand human language directly. It only understands machine-level instructions. Programming languages act as a bridge between humans and computers.

A set of instructions written to perform a specific task is called a **Program**.

Real-Time Examples

- ATM Withdrawal
- Online Shopping
- Food Delivery Apps
- Instagram Feed Loading
- YouTube Video Recommendations
- Uber Ride Booking

All these applications work because programmers write instructions that computers can execute.

2. What is a Programming Language?

A Programming Language is a language used to communicate with computers and create software applications.

Examples:

- Python
- Java
- C
- C++
- JavaScript
- C#
- Go
- Rust

3. Types of Programming Paradigms

Programming languages generally follow different programming styles called paradigms.

The two most common paradigms are:

1. Procedural Programming
2. Object-Oriented Programming (OOP)

4. Procedural Programming Language

Procedural Programming focuses on Functions and Procedures.

The program is divided into small functions, and execution happens step-by-step.

The main focus is:

- Functions
- Logic
- Sequence of Steps

Examples:

- C
- Pascal
- BASIC

Real-Time Example

Food Ordering Process:

Step 1 → Login

Step 2 → Select Restaurant

Step 3 → Select Food

Step 4 → Payment

Step 5 → Order Confirmation

Each step can be written as a separate function.

None

```
login()
```

```
select_restaurant()  
select_food()  
payment()  
confirm_order()
```

This is Procedural Programming because execution happens step by step.

Characteristics of Procedural Programming

- Function Based
- Top to Bottom Execution
- Easier for Small Applications
- Less Code Reusability
- Data is less secure
- Difficult to manage large projects

Advantages of Procedural Programming

- Easy to Learn
- Easy to Write
- Fast Execution
- Suitable for Small Programs

Disadvantages of Procedural Programming

- Difficult for Large Projects
- Data Security Issues
- Less Reusability
- Hard to Maintain

5. Object-Oriented Programming (OOP)

Object-Oriented Programming organizes code using Objects and Classes.

Instead of focusing only on functions, OOP focuses on real-world entities.

Examples:

- Customer
- Product
- Employee
- Bank Account

Everything is represented as an Object.

Real-Time Example

Amazon Product

Product Properties:

- Product Name
- Price
- Rating
- Stock

Product Behaviors:

- Add to Cart
- Buy Now
- Add Review

In OOP:

Python

Product Object

Attributes:

name
price
rating

Methods:

add_to_cart()
buy_now()
add_review()

Characteristics of OOP

- Class Based
- Object Based
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction
- Reusability
- Security

Advantages of OOP

- Reusable Code
- Better Security
- Easy Maintenance
- Easy Scalability
- Suitable for Large Applications

Disadvantages of OOP

- More Memory Usage
- Slightly Slower Execution
- More Complex for Beginners

Procedural vs Object-Oriented Programming

Feature	Procedural Programming	OOP
Focus	Functions	Objects
Structure	Function Based	Class Based
Reusability	Low	High
Security	Low	High
Maintenance	Difficult	Easy
Scalability	Low	High
Memory Usage	Less	More
Suitable For	Small Applications	Large Applications

6. What is Python?

Python is a high-level, interpreted, general-purpose programming language created to make programming simple and readable.

Python emphasizes:

- Simplicity
- Readability
- Productivity

Python code often looks similar to English sentences.

Example:

```
Python
name = "Ravi"
print(name)
```

Features of Python

- Easy Syntax
- High Level Language
- Interpreted Language
- Object-Oriented
- Open Source
- Portable
- Dynamically Typed
- Large Community Support
- Huge Library Ecosystem

7. Why Python?

Python is one of the most popular programming languages because it requires fewer lines of code and is easy to learn.

Example:

Printing Hello World

```
Python
print("Hello World")
```

Why Companies Use Python

- Faster Development
- Less Code
- Easy Maintenance
- Large Ecosystem
- Strong Community

Companies using Python:

- Google
- Netflix
- Instagram
- Spotify
- Dropbox

8. History of Python

Python was created by:

Guido van Rossum

Development Started:

1989

First Release:

1991

Name Origin:

The name "Python" was inspired by the comedy show:

Monty Python's Flying Circus

Python was not named after the snake.

Python Version Timeline

Version	Year
Python 1.0	1994
Python 2.0	2000
Python 3.0	2008
Python 3.x	Current Series

Python 3 is currently the standard version.

9. Applications of Python

Web Development

Examples:

- Django
- Flask
- FastAPI

Used for:

- Websites
- APIs
- Backend Systems

Data Science

Used for:

- Data Analysis
- Data Visualization
- Business Analytics

Libraries:

- NumPy
- Pandas
- Matplotlib

Artificial Intelligence

Used for:

- Machine Learning
- Deep Learning
- Computer Vision
- NLP

Libraries:

- TensorFlow
- PyTorch
- Scikit-Learn

Automation

Used for:

- Excel Automation
- Email Automation
- Web Scraping
- File Handling

Cyber Security

Used for:

- Network Scanning
- Penetration Testing
- Security Automation

Game Development

Libraries:

- Pygame

Examples:

- 2D Games
- Educational Games

Desktop Applications

Libraries:

- Tkinter
- PyQt

Examples:

- Calculator
- Billing Software
- Management Systems

10. Memory Allocation: Procedural vs OOP

Understanding memory allocation is important because it explains how data is stored and managed.

Procedural Programming Memory Allocation

In Procedural Programming:

- Memory mainly stores variables and functions.
- Functions are called one after another.
- Data is usually shared globally or passed between functions.
- Memory usage is relatively low.

Example:

Python

```
name = "Ravi"  
price = 500  
  
calculate_bill()  
print_bill()
```

Memory:

None

Stack Memory

```
name
price
```

Function Memory

```
calculate_bill()
print_bill()
```

Since there are only variables and functions, memory consumption is less.

OOP Memory Allocation

In OOP:

Objects are created in Heap Memory.

Each object stores:

- Attributes
- Methods Reference

Example:

None

Product Object

```
name = Laptop
price = 50000
rating = 4.5
```

Memory:

None

Heap Memory

Product Object

```
name
price
rating
```

Stack Memory

product1 → Object Reference

Objects consume more memory because every object stores its own state (attributes).

Procedural vs OOP Memory Allocation

Feature	Procedural	OOP
Memory Usage	Less	More
Stores	Variables + Functions	Objects + Attributes
Object Creation	Not Required	Required
Heap Usage	Minimal	Heavy
Reusability	Low	High
Security	Lower	Higher
Suitable For	Small Programs	Large Applications